

Cifrado

El cifrado de información es el proceso de convertir datos *comprensibles* en una forma *incomprensible*. Para que esto pueda llevarse a cabo, la información original es transformada por la aplicación de un algoritmo específicamente diseñado para producir una salida cifrada que dificulte su reconstrucción.

El objetivo principal del cifrado es proteger la *confidencialidad* de la información, con el propósito de que *solo las personas autorizadas puedan acceder y comprender los datos*.

El cifrado de información *se basa en el uso de una clave*, que es un *valor secreto o conocido únicamente por las partes autorizadas*. La clave se utiliza como un parámetro en el algoritmo de cifrado para transformar los datos originales en su equivalente cifrado. Sin la clave correcta, es extremadamente difícil o prácticamente imposible¹ recuperar la información original de manera significativa a partir del texto cifrado.

Existen dos tipos principales de cifrado utilizados en criptografía:

1. **Cifrado simétrico, reversible o de clave secreta:** En este tipo de cifrado, *la misma clave se utiliza tanto para el proceso de cifrado como para el de descifrado*. Tanto el remitente como el receptor deben conocer y mantener en secreto la misma clave. Los algoritmos de cifrado simétrico son rápidos y eficientes, lo que los hace adecuados para el cifrado y descifrado de grandes volúmenes de datos. Ejemplos de algoritmos de cifrado simétrico son AES, DES y 3DES, Blowfish, Twofish, etc.
2. **Cifrado asimétrico, irreversible o de clave pública:** En este tipo de cifrado, *se utilizan dos claves diferentes pero relacionadas: una clave pública y una clave privada*. La clave pública se utiliza para cifrar los datos, mientras que la clave privada se utiliza para descifrarlos. La clave pública se puede compartir libremente, mientras que la clave privada debe mantenerse en secreto. Los algoritmos de cifrado asimétrico se utilizan para establecer canales seguros de comunicación, autenticación y firma digital. Ejemplos de algoritmos de cifrado asimétrico son RSA, ECC, DSA, ElGamal, etc.

Además de la confidencialidad, el cifrado también puede ofrecer otros beneficios, como la autenticación de los datos y la integridad de la información. Al combinar el cifrado con otras técnicas criptográficas, como las firmas digitales o los algoritmos de hash, se pueden lograr niveles más altos de seguridad en la protección de la información.

Cifrado simétrico

El cifrado simétrico, también conocido como cifrado de clave secreta o cifrado de clave compartida, es un método de cifrado en el que se utiliza una única clave para tanto el proceso de cifrado como el de descifrado de los datos. La misma clave secreta se comparte entre el remitente y el receptor de la información.

La lógica de funcionamiento es la siguiente: los datos comprensibles se transforman en su versión incomprensible (cifrada) utilizando la clave secreta. Luego, el receptor utiliza la misma clave secreta para revertir el proceso y recuperar los datos originales.

Las características principales del cifrado simétrico son:

1. **Uso de una única clave:** Tanto el remitente como el receptor utilizan la misma clave para cifrar y descifrar los datos. Esto implica que la clave debe mantenerse en secreto y compartirse de manera segura entre las partes autorizadas.

¹ En teoría, con los algoritmos de cifrado adecuados y una implementación correcta, descifrar un texto encriptado sin conocer la clave apropiada se considera **computacionalmente impracticable por el costo de fuerza bruta**. Esto se conoce como **"criptografía segura"** y es un objetivo fundamental en el diseño de los algoritmos de cifrado modernos.

2. **Eficiencia y velocidad:** Los algoritmos de cifrado simétrico suelen ser rápidos y eficientes, lo que los hace adecuados para el cifrado y descifrado de grandes volúmenes de datos. Estos algoritmos están diseñados para proporcionar un alto rendimiento computacional.
3. **Aplicaciones comunes:** El cifrado simétrico se utiliza en una amplia gama de aplicaciones, como la protección de comunicaciones seguras, el almacenamiento de datos sensibles, la encriptación de archivos y la autenticación de mensajes.

Es importante destacar que la seguridad en el cifrado simétrico depende en gran medida de la seguridad y confidencialidad de la clave compartida. El desafío principal del cifrado simétrico radica en el intercambio seguro de la clave entre el remitente y el receptor, ya que cualquier persona que tenga acceso a la clave puede descifrar los datos.

Ejemplo de utilización del algoritmo simétrico AES en JavaScript²

JS/ECMAScript 2017

```
// Texto para encriptar
const textoOriginal = 'Hola, mundo!';

// Convertir el texto en una matriz de bytes
const textoCodificado = new TextEncoder().encode(textoOriginal);

// Función para generar una clave aleatoria
async function generarClave()
{
    return await window.crypto.subtle.generateKey(
        {
            name: 'AES-GCM',
            length: 256,
        },
        true,
        ['encrypt', 'decrypt']
    );
}

// Función para encriptar los datos
async function encriptarDatos(clave)
{
    const textoEncriptado = await window.crypto.subtle.encrypt(
        {
            name: 'AES-GCM',
            iv: window.crypto.getRandomValues(new Uint8Array(12)),
        },
        clave,
        textoCodificado
    );
    return textoEncriptado;
}

// Función para desencriptar los datos
async function desencriptarDatos(clave, textoEncriptado)
{
    const textoDesencriptado = await window.crypto.subtle.decrypt(
        {
            name: 'AES-GCM',
            iv: window.crypto.getRandomValues(new Uint8Array(12)),
        },
        clave,
        textoEncriptado
    );
    return textoDesencriptado;
}
```

² SubtleCrypto WebAPI : <https://developer.mozilla.org/es/docs/Web/API/SubtleCrypto>

```
// Función principal asincrónica
async function main()
{
    try
    {
        // Generar clave
        const clave = await generarClave();

        // Encriptar datos
        const textoEncriptado = await encriptarDatos(clave);

        console.log('Texto encriptado:', new Uint8Array(textoEncriptado));

        // Desencriptar datos
        const textoDesencriptado = await desencriptarDatos(clave, textoEncriptado);

        const textoDecodificado = new TextDecoder().decode(new Uint8Array(textoDesencriptado));
        console.log('Texto desencriptado:', textoDecodificado);
    }
    catch (error)
    {
        console.error(error);
    }
}

// Llamar a la función principal
main();
```

Análisis de fortalezas y debilidades³

	Cifrado simétrico	Cifrado asimétrico
Fortalezas	Eficiencia: Los cifrados simétricos son generalmente más rápidos y eficientes en términos de rendimiento computacional. Escalabilidad: Son adecuados para aplicaciones que involucran una comunicación directa entre un número limitado de participantes. Protección de datos a largo plazo: Pueden proporcionar una protección a largo plazo de los datos cifrados, ya que la clave secreta no se comparte o expone.	Seguridad en el intercambio de claves: El cifrado asimétrico permite un intercambio seguro de claves utilizando claves públicas y privadas. Escalabilidad: Son adecuados para aplicaciones con múltiples participantes, ya que no se requiere una clave compartida entre todos los participantes. Autenticación y firma digital: Los cifrados asimétricos permiten la autenticación y verificación de la integridad de los datos utilizando firmas digitales.
Debilidades	Intercambio seguro de claves: El mayor desafío es el intercambio seguro de la clave secreta entre el remitente y el receptor. Escalabilidad de claves: Cada par de remitente y receptor debe tener su propia clave compartida, lo que puede volverse complejo y poco práctico en escenarios con muchos participantes. Seguridad en caso de pérdida de clave: Si se pierde o se compromete la clave compartida, todos los datos cifrados con esa clave se vuelven inaccesibles.	Rendimiento computacional: Los cifrados asimétricos son más lentos y requieren más recursos computacionales en comparación con los cifrados simétricos. Protección de datos a largo plazo: Los cifrados asimétricos pueden enfrentar problemas de seguridad si se descubren vulnerabilidades en los algoritmos o en la clave privada. Uso de claves públicas: La administración adecuada de las claves públicas es esencial para garantizar la seguridad y la autenticidad de las comunicaciones.

³ Es importante tener en cuenta que estos son solo aspectos generales y las fortalezas y debilidades pueden variar dependiendo de los algoritmos y las implementaciones específicas utilizadas en cada caso. Además, los requisitos de seguridad, rendimiento y contexto de la aplicación deben considerarse al seleccionar el método de cifrado más adecuado para una situación particular. En muchos casos, se utilizan enfoques híbridos que combinan tanto cifrados simétricos como asimétricos para aprovechar las fortalezas de ambos y compensar sus debilidades.

Funciones hash criptográficas

Las funciones hash criptográficas son funciones que toman una *entrada de datos de cualquier longitud* y generan un valor único *de longitud fija*, llamado *hash* o *resumen*. Tienen como propósito *verificar la integridad de los datos* y como derivación de éste, verificar la *autenticidad* y la *detección de la existencia de cambios no autorizados*. A diferencia de las funciones de hash convencionales, poseen propiedades específicas de seguridad y confiabilidad. Están diseñadas para ser seguras contra ataques criptoanalíticos, resistentes a colisiones y se utilizan en protocolos de seguridad como las firmas digitales, la generación de llaves, la autenticación de mensajes, entre otras.

Las funciones hash criptográficas tienen varias propiedades clave que son fundamentales para su uso en aplicaciones criptográficas. Estas propiedades incluyen:

1. **Unicidad:** Cada *conjunto diferente de datos de entrada* a la función debe producir *siempre un valor hash único*.
2. **Resistencia a las colisiones:** Cuanto mejor sea la calidad de la función de hash criptográfica, menor es la probabilidad de que *conjuntos diferentes de datos produzcan el mismo valor de hash* y por ende, más resistente a las colisiones.
3. **Sensibilidad a los cambios:** El cambio de un solo bit en el conjunto de datos de entrada debe producir un hash totalmente diferente al que produce el conjunto original, de forma tal que no pueda establecerse un patrón de cambio y la generación de hash se mantenga *impredecible*.
4. **Determinismo:** Dados *los mismos datos de entrada*, una función hash criptográfica siempre debe producir el *mismo hash de salida*. Esto es esencial para verificar la integridad de los datos y realizar comparaciones de hashes.
5. **Resistencia a la inversión:** Las funciones hash criptográficas deben ser computacionalmente impracticables de invertir, lo que significa que no se debe poder recuperar los datos originales a partir del hash. Esto garantiza que los hashes sean *unidireccionales* y protege la *confidencialidad* de los datos.
6. **Eficiencia computacional:** Las funciones hash criptográficas deben ser rápidas y eficientes en términos de tiempo y recursos computacionales. Deben poder calcular el hash rápidamente, incluso para grandes volúmenes de datos, sin comprometer la seguridad.

Ejemplos de funciones hash criptográficas más utilizadas

SHA-256 (Secure Hash Algorithm 256 bits): Es parte de la familia de funciones de hash SHA-2 y genera un hash de 256 bits. SHA-256 es ampliamente utilizado y se considera seguro para aplicaciones criptográficas en la actualidad.

SHA-3 (Secure Hash Algorithm 3): Es una familia de algoritmos de hash criptográficos seleccionados como resultado del proceso de competencia llamado "Keccak". SHA-3 ofrece una variedad de tamaños de hash, como SHA-3-256 y SHA-3-512, y se considera seguro y resistente a los ataques conocidos.

MD5 (Message Digest 5): Aunque se ha utilizado ampliamente en el pasado, MD5 se considera ⁴seguro para aplicaciones criptográficas debido a su vulnerabilidad a colisiones y ataques de fuerza bruta. Actualmente, su uso se limita principalmente a aplicaciones no criptográficas, como la verificación de integridad de archivos.

⁴ A medida que avanzan los avances en la criptoanálisis y la computación, los algoritmos de hash criptográficos pueden volverse vulnerables con el tiempo. Por lo tanto, es esencial seguir las recomendaciones y las actualizaciones de los expertos en criptografía y utilizar algoritmos de hash criptográficos actualizados y seguros.

Código de ejemplo de utilización del algoritmo SHA256 en JavaScript

JS/ECMAScript 2017

```
// Función para calcular el hash SHA-256 de una cadena
async function calcularHashSHA256(cadena)
{
    const encoder = new TextEncoder();
    const data = encoder.encode(cadena);

    const hashBuffer = await crypto.subtle.digest('SHA-256', data);
    const hashArray = Array.from(new Uint8Array(hashBuffer));
    const hashHex = hashArray.map(byte => byte.toString(16).padStart(2, '0')).join('');

    return hashHex;
}

// Función principal asincrónica
async function main()
{
    try
    {
        const cadenaOriginal = 'Hola, mundo!';

        // Calcular el hash SHA-256 de la cadena
        const hash = await calcularHashSHA256(cadenaOriginal);

        console.log('Hash SHA-256:', hash);
    }
    catch (error)
    {
        console.error(error);
    }
}

// Llamar a la función principal
main();
```

Queda a cargo del estudiante el desarrollo de una abstracción conveniente y adecuada para el trabajo con funciones hash criptográficas.

Diferencias entre el cifrado de información y el uso de funciones de hash criptográficas

El cifrado y las funciones de hash criptográficas comparten algunos aspectos comunes, pero también tienen diferencias importantes. Aquí hay algunas similitudes y puntos en común entre el cifrado y las funciones de hash criptográficas:

1. **Seguridad:** Tanto el cifrado como las funciones de hash criptográficas están diseñados para garantizar la seguridad de la información. El cifrado se enfoca en la confidencialidad de los datos, mientras que las funciones de hash se utilizan para la integridad y autenticidad de los datos.
2. **Aplicaciones criptográficas:** Tanto el cifrado como las funciones de hash criptográficas se utilizan ampliamente en aplicaciones criptográficas para proteger la información, garantizar la autenticidad y detectar modificaciones no autorizadas.
3. **Claves y parámetros:** En algunos casos, tanto el cifrado como las funciones de hash pueden requerir claves o parámetros adicionales para su funcionamiento. Por ejemplo, los algoritmos de cifrado simétrico utilizan claves para cifrar y descifrar los datos, mientras que algunas funciones de hash pueden requerir un parámetro adicional, como una sal (salt) para aumentar la seguridad.

Sin embargo, también existen diferencias significativas entre el cifrado y las funciones de hash criptográficas. El cifrado se enfoca en la confidencialidad y utiliza claves para transformar los datos legibles en datos ininteligibles y viceversa. Por otro lado, las funciones de hash se centran en la integridad y autenticidad de los datos, generando un resumen único del conjunto de datos de entrada. Además, las funciones de hash son unidireccionales y no pueden recuperar los datos originales a partir del hash, mientras que el cifrado permite el descifrado utilizando la clave correspondiente.

Aunque el cifrado y las funciones de hash criptográficas comparten ciertos aspectos comunes y se utilizan en aplicaciones criptográficas, sus objetivos y propiedades son distintos. Cada uno tiene su propio propósito y se utiliza en diferentes contextos para garantizar la seguridad de la información.

1. **Unidireccionalidad:** Los algoritmos de hash son unidireccionales, lo que significa que no es posible recuperar los datos originales a partir del hash. El proceso de hash es irreversible, lo que impide descifrar la información original. En contraste, los algoritmos de cifrado permiten tanto el cifrado como el descifrado utilizando una clave.
2. **Clave:** Los algoritmos de cifrado requieren una clave para convertir los datos originales en texto cifrado y viceversa. En cambio, los algoritmos de hash no utilizan una clave para generar el hash. Esto implica que no hay una forma de proteger la información confidencial utilizando una clave secreta.
3. **Tamaño fijo:** Los algoritmos de hash generan un hash de tamaño fijo, independientemente del tamaño de los datos de entrada. Esto significa que la salida del hash siempre tendrá la misma longitud, lo que dificulta el cifrado y descifrado de datos de longitud variable.
4. **No confidencialidad:** Los algoritmos de hash no garantizan la confidencialidad de los datos, ya que el hash se puede calcular fácilmente a partir de los datos originales sin necesidad de una clave. Esto significa que cualquiera que tenga acceso a los datos puede generar el hash correspondiente sin dificultad.